

Exercício Prático — Testes Manuais de API

Sistema: Banking API Training System

Objetivo: Praticar elaboração de casos de teste, execução manual, registro de bugs e análise de cobertura

Sobre Este Exercício

Você receberá a **especificação de requisitos** de uma API bancária simples e deverá:

1. Elaborar 10 casos de teste cobrindo diferentes funcionalidades
2. Criar um plano de teste documentando estratégia e escopo
3. Executar os testes manualmente e documentar resultados
4. Registrar bugs encontrados seguindo boas práticas
5. Calcular a cobertura de testes por requisitos

Este exercício simula um cenário real de trabalho em QA, onde você recebe uma especificação e precisa garantir a qualidade do sistema através de testes manuais.

Como Subir o Sistema

Pré-requisitos

Antes de começar, certifique-se de ter instalado:

- **Node.js** versão 18 ou superior (Download: <https://nodejs.org/>)
- Um navegador web (Chrome, Firefox ou Edge)
- Um cliente de API (escolha um):
 - **Postman** (Download: <https://www.postman.com/downloads/>)
 - **Insomnia** (Download: <https://insomnia.rest/download>)
 - Extensão **Thunder Client** no VS Code

Passo a Passo

1. Instalar as dependências do backend

Abra o terminal na pasta do projeto e execute:

```
cd backend  
npm install
```

Aguarde a instalação das dependências (express, cors, swagger, etc.).

2. Iniciar o servidor

Ainda no terminal, dentro da pasta backend, execute:

```
npm run dev
```

Você verá uma mensagem similar a:

Servidor rodando em <http://localhost:3000>
Swagger disponível em <http://localhost:3000/api-docs>

Pronto! A API está rodando.

3. Verificar se está funcionando

Abra o navegador e acesse:

- **API:** <http://localhost:3000>
- **Swagger (documentação interativa):** <http://localhost:3000/api-docs>

Se ambos carregarem, o sistema está pronto para os testes.

4. (Opcional) Abrir os frontends

Os frontends **não precisam de servidor** — basta abrir os arquivos HTML:

- **Frontend Web:** Abrir frontend-web/index.html no navegador
- **Frontend Mobile:** Abrir frontend-mobile/index.html no navegador

Especificação de Requisitos — Banking API

Visão Geral

O Banking API é um sistema bancário simplificado que permite:

- Criar, consultar, atualizar e deletar contas bancárias
- Realizar depósitos (adicionar dinheiro à conta)
- Realizar transferências entre contas

O sistema armazena dados **em memória** (ao reiniciar o servidor, os dados são apagados).

Requisitos Funcionais

RF-001: Criar Conta

Descrição: O sistema deve permitir a criação de uma nova conta bancária.

Endpoint: POST /accounts

Payload (JSON):

```
{
  "name": "João Silva",
  "email": "joao@email.com",
  "cpf": "111.111.111-11",
  "initialBalance": 500
}
```

Regras de Negócio:

- Todos os campos são obrigatórios
- initialBalance deve ser maior ou igual a 0 (não pode ser negativo)
- cpf deve ser único (não pode haver duas contas com o mesmo CPF)

Respostas Esperadas:

- **201 Created:** Conta criada com sucesso, retorna objeto completo com id, balance, createdAt
 - **400 Bad Request:** Campos ausentes, initialBalance negativo ou CPF duplicado
-

RF-002: Listar Todas as Contas

Descrição: O sistema deve retornar a lista de todas as contas cadastradas.

Endpoint: GET /accounts

Regras de Negócio:

- Retorna array vazio [] se não houver contas
- Retorna array de objetos conta se houver contas cadastradas

Respostas Esperadas:

- **200 OK:** Lista de contas (pode ser vazia)
-

RF-003: Buscar Conta por ID

Descrição: O sistema deve retornar os dados de uma conta específica pelo ID.

Endpoint: GET /accounts/{id}

Regras de Negócio:

- ID deve existir no sistema

Respostas Esperadas:

- **200 OK:** Retorna objeto da conta encontrada
 - **404 Not Found:** Conta não encontrada (ID inexistente)
-

RF-004: Atualizar Dados da Conta

Descrição: O sistema deve permitir atualizar nome e email de uma conta.

Endpoint: PUT /accounts/{id}

Payload (JSON):

```
{  
  "name": "João da Silva Junior",  
  "email": "joao.junior@email.com"  
}
```

Regras de Negócio:

- Pode atualizar name e/ou email (campos opcionais)
- **NÃO** pode alterar cpf nem balance

Respostas Esperadas:

- **200 OK:** Conta atualizada com sucesso
 - **404 Not Found:** Conta não encontrada
-

RF-005: Deletar Conta

Descrição: O sistema deve permitir remover uma conta do sistema.

Endpoint: DELETE /accounts/{id}

Regras de Negócio:

- Conta deve existir para ser removida

Respostas Esperadas:

- **200 OK:** { "message": "Conta removida com sucesso" }
 - **404 Not Found:** Conta não encontrada
-

RF-006: Realizar Depósito

Descrição: O sistema deve permitir adicionar dinheiro a uma conta.

Endpoint: POST /accounts/{id}/deposit

Payload (JSON):

```
{  
  "amount": 200  
}
```

Regras de Negócio:

- amount deve ser maior que 0 (não pode depositar valor zero ou negativo)
- Conta deve existir
- Saldo final = saldo atual + amount

Respostas Esperadas:

- **200 OK:** { "message": "Depósito realizado", "balance": 700 }
 - **400 Bad Request:** amount inválido (≤ 0)
 - **404 Not Found:** Conta não encontrada
-

RF-007: Realizar Transferência

Descrição: O sistema deve permitir transferir dinheiro entre duas contas.

Endpoint: POST /accounts/{id}/transfer

Payload (JSON):

```
{  
  "targetId": "uuid-conta-destino",  
  "amount": 100  
}
```

Regras de Negócio:

- Conta de origem (ID na URL) deve existir
- Conta de destino (targetId) deve existir
- amount deve ser maior que 0
- Conta origem deve ter saldo suficiente ($\text{saldo} \geq \text{amount}$)
- Origem e destino **não podem ser a mesma conta**

Respostas Esperadas:

- **200 OK:** { "message": "Transferência realizada com sucesso", "sourceBalance": 400, "targetBalance": 300 }

- **400 Bad Request:** Saldo insuficiente, amount inválido, ou origem = destino
 - **404 Not Found:** Conta origem ou destino não encontrada
-

Parte 1: Elaborar 10 Casos de Teste

O que é um Caso de Teste?

Um **caso de teste** é um documento que descreve:

- **O que testar** (funcionalidade específica)
- **Como testar** (passos detalhados)
- **Resultado esperado** (o que deve acontecer)

Template de Caso de Teste

Use o seguinte formato para cada caso de teste:

Campo	Descrição
ID	Identificador único (ex: CT-001)
Título	Resumo do que será testado
Pré-condições	Estado do sistema antes do teste
Passos	Sequência de ações a executar
Resultado Esperado	O que deve acontecer
Requisito Relacionado	RF-XXX

Exemplo de Caso de Teste

ID: CT-001

Título: Criar conta com dados válidos

Pré-condições: API rodando, nenhuma conta com CPF "123.456.789-00" cadastrada

Passos:

1. Abrir Postman/Insomnia
2. Criar requisição POST `http://localhost:3000/accounts`
3. Adicionar header `Content-Type: application/json`
4. Adicionar body:

```
{  
  "name": "Maria Santos",  
  "email": "maria@email.com",
```

```
"cpf": "123.456.789-00",  
"initialBalance": 1000  
}
```

5. Enviar requisição

Resultado Esperado:

- Status: 201 Created
- Response contém:
 - id (UUID gerado)
 - name: "Maria Santos"
 - email: "maria@email.com"
 - cpf: "123.456.789-00"
 - balance: 1000
 - createdAt (data/hora atual)

Requisito Relacionado: RF-001

Seu Desafio

Elabore **10 casos de teste** seguindo o template acima. Dicas:

Cobertura sugerida:

- 2 casos para criação de conta (1 válido + 1 inválido)
- 1 caso para listar contas
- 1 caso para buscar conta por ID
- 1 caso para atualizar conta
- 1 caso para deletar conta
- 2 casos para depósito (1 válido + 1 inválido)
- 2 casos para transferência (1 válido + 1 inválido)

Dica: Teste tanto os **caminhos felizes** (tudo funciona) quanto os **caminhos de erro** (validações, dados inválidos).

Parte 2: Elaborar o Plano de Teste

O que é um Plano de Teste?

Um **plano de teste** é um documento estratégico que define:

- **Escopo** — o que será testado (e o que não será)
- **Abordagem** — como os testes serão executados
- **Recursos** — ferramentas e ambiente necessários
- **Cronograma** — quando os testes acontecerão
- **Critérios de aceite** — quando considerar "pronto"

Template de Plano de Teste

Use a seguinte estrutura:

1. Identificação

- **Projeto:** Banking API Training System
- **Versão:** 1.0
- **Responsável:** [Seu nome]
- **Data:** [Data atual]

2. Escopo

O que será testado:

- Criar conta (RF-001)
- Listar contas (RF-002)
- Buscar conta por ID (RF-003)
- Atualizar conta (RF-004)
- Deletar conta (RF-005)
- Depósito (RF-006)
- Transferência (RF-007)

O que NÃO será testado:

- Performance (carga, stress)
- Segurança (autenticação, autorização)
- Frontends (apenas API)

3. Abordagem

- **Tipo de teste:** Teste funcional manual de API
- **Ferramentas:** Postman/Insomnia
- **Ambiente:** Local (<http://localhost:3000>)
- **Dados de teste:** Massas criadas durante execução

4. Recursos Necessários

- Node.js instalado

- Servidor backend rodando
- Cliente de API (Postman/Insomnia)
- Documentação Swagger (<http://localhost:3000/api-docs>)

5. Cronograma

Atividade	Duração Estimada
Elaboração dos casos de teste	1h
Configuração do ambiente	15min
Execução dos testes	1h
Registro de bugs	30min
Relatório final	30min

6. Critérios de Aceite

Critérios de sucesso:

- Todos os requisitos funcionais possuem pelo menos 1 caso de teste
- Casos de teste cobrem caminhos felizes e de erro
- Todos os casos foram executados
- Bugs críticos foram registrados
- Cobertura de requisitos $\geq 80\%$

Critério de bloqueio:

Se algum endpoint crítico (criar conta, depósito, transferência) não funcionar, os testes são suspensos até correção.

Seu Desafio

Preencha o plano de teste acima, adaptando:

- Seu nome como responsável
- Data atual
- Estimativas de tempo realistas para você
- Qualquer observação adicional relevante

Parte 3: Executar os Testes e Criar Relatório de Execução

O que é um Relatório de Execução?

É um documento que registra o **resultado real** da execução dos casos de teste, incluindo:

- Status (passou, falhou, bloqueado)
- Evidências (prints, logs)
- Observações sobre comportamentos inesperados

Template de Relatório de Execução

ID	Título	Status	Observações	Evidência
CT-001	Criar conta válida	Passou	Conta criada corretamente	Print response
CT-002	Criar conta CPF duplicado	Falhou	Retornou 201 ao invés de 400	Bug #001
CT-003	Listar contas	Passou	Retornou array com 2 contas	-

Legenda de Status:

- **Passou** — Resultado obtido = Resultado esperado
- **Falhou** — Resultado obtido $\neq$ Resultado esperado (bug encontrado)
- **Bloqueado** — Não foi possível executar (dependência quebrada)
- **Não Executado** — Ainda não foi testado

Exemplo de Registro de Execução

CT-001: Criar conta com dados válidos

Executado em: 23/02/2026 às 22:30

Executado por: Ana Silva

Passos executados:

1. Aberto Postman
2. Criado POST <http://localhost:3000/accounts>
3. Adicionado header Content-Type
4. Adicionado body com dados válidos
5. Enviado requisição

Resultado obtido:

- Status: **201 Created**
- Response:
{
 "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",

```
"name": "Maria Santos",
"email": "maria@email.com",
"cpf": "123.456.789-00",
"balance": 1000,
"createdAt": "2026-02-23T22:30:45.123Z"
}
```

Comparação:

- Status esperado: 201 ✓
- Campos presentes: Todos ✓
- Valores corretos: Sim ✓

Conclusão: PASSOU

Seu Desafio

Execute cada um dos 10 casos de teste que você criou e preencha o relatório de execução com:

1. Data e hora da execução
2. Status (passou/falhou/bloqueado)
3. Observações sobre o comportamento
4. Se falhou, referencie o bug registrado
5. (Opcional) Print da tela como evidência

Dica: Use o Postman Collections para salvar suas requisições e facilitar a reexecução.

Parte 4: Registrar Bugs Encontrados

O que é um Registro de Bug?

É um documento que descreve um **comportamento incorreto** do sistema de forma clara e completa para que o desenvolvedor possa reproduzir e corrigir.

Template de Registro de Bug

Campo	Descrição
ID	Identificador único (ex: BUG-001)
Título	Resumo do problema (máximo 80 caracteres)
Severidade	Crítica / Alta / Média / Baixa
Prioridade	Alta / Média / Baixa
Ambiente	Local, Dev, Staging, Prod

Versão	Versão do sistema onde foi encontrado
Pré-condições	Estado do sistema antes do bug
Passos para Reproduzir	Sequência detalhada para replicar
Resultado Esperado	O que deveria acontecer
Resultado Obtido	O que realmente aconteceu
Evidências	Prints, logs, vídeos
Observações	Informações adicionais

Classificação de Severidade

Severidade	Descrição	Exemplo
Crítica	Sistema quebrado, funcionalidade principal indisponível	API não inicia, todas as requisições retornam 500
Alta	Funcionalidade importante não funciona	Transferência não debita da conta origem
Média	Funcionalidade funciona parcialmente ou com workaround	Mensagem de erro genérica ao invés de específica
Baixa	Problema cosmético ou de usabilidade	Texto com erro de ortografia

Exemplo de Registro de Bug

ID: BUG-001

Título: Sistema aceita criação de conta com saldo inicial negativo

Severidade: Alta

Prioridade: Alta

Ambiente: Local (<http://localhost:3000>)

Versão: 1.0

Pré-condições:

- API rodando
- Nenhuma conta previamente criada

Passos para Reproduzir:

1. Abrir Postman
2. Criar requisição POST <http://localhost:3000/accounts>
3. Adicionar body:


```
{
  "name": "Teste Negativo",
  "email": "teste@email.com",
```

```
"cpf": "999.999.999-99",  
"initialBalance": -500  
}
```

4. Enviar requisição

Resultado Esperado:

- Status: 400 Bad Request
- Mensagem de erro: "initialBalance não pode ser negativo" ou similar

Resultado Obtido:

- Status: 201 Created
- Conta criada com balance: -500

Evidências:

```
{  
  "id": "abc-123",  
  "name": "Teste Negativo",  
  "balance": -500,  
  ...  
}
```

Observações:

- A validação de `initialBalance >= 0` não está funcionando
- Isso viola a regra de negócio do RF-001
- Impacto: usuários podem criar contas com dívida inicial, o que não faz sentido para o negócio

Seu Desafio

Para cada caso de teste que **falhou** durante a execução:

1. Crie um registro de bug detalhado seguindo o template
2. Classifique corretamente a severidade
3. Descreva os passos para reproduzir de forma clara
4. Anexe evidências (prints, JSONs de resposta)

Dica: Um bom registro de bug economiza tempo do desenvolvedor e acelera a correção.

Parte 5: Calcular a Cobertura de Testes

O que é Cobertura de Testes?

Cobertura de testes mede o percentual de requisitos que foram testados em relação ao total de requisitos do sistema.

Fórmula:

$$\text{Cobertura (\%)} = \frac{\text{Requisitos Testados}}{\text{Total de Requisitos}} \times 100$$

Como Calcular

Passo 1: Listar todos os requisitos

O Banking API possui 7 **requisitos funcionais**:

Requisito	Descrição
RF-001	Criar conta
RF-002	Listar contas
RF-003	Buscar conta por ID
RF-004	Atualizar conta
RF-005	Deletar conta
RF-006	Realizar depósito
RF-007	Realizar transferência

Total de Requisitos: 7

Passo 2: Verificar quais requisitos foram testados

Revise seus 10 casos de teste e marque quais requisitos foram cobertos:

Requisito	Casos de Teste	Status
RF-001	CT-001, CT-002	Testado
RF-002	CT-003	Testado
RF-003	CT-004	Testado
RF-004	CT-005	Testado
RF-005	CT-006	Testado
RF-006	CT-007, CT-008	Testado
RF-007	CT-009, CT-010	Testado

Requisitos Testados: 7

Passo 3: Calcular o percentual

$$\text{Cobertura} = \frac{7}{7} \times 100 = 100\%$$

Matriz de Rastreabilidade

Uma **matriz de rastreabilidade** conecta requisitos → casos de teste → resultados.

Exemplo:

Requisito	Descrição	Casos de Teste	Resultado	Bugs
RF-001	Criar conta	CT-001, CT-002	1 passou, 1 falhou	BUG-001
RF-002	Listar contas	CT-003	Passou	-
RF-003	Buscar por ID	CT-004	Passou	-
RF-004	Atualizar conta	CT-005	Passou	-
RF-005	Deletar conta	CT-006	Passou	-
RF-006	Depósito	CT-007, CT-008	2 passaram	-
RF-007	Transferência	CT-009, CT-010	1 passou, 1 falhou	BUG-002

Análise:

- **Cobertura:** 100% (todos os requisitos possuem pelo menos 1 caso)
- **Taxa de sucesso:** 80% (8 de 10 casos passaram)
- **Bugs encontrados:** 2
- **Requisitos com bugs:** RF-001, RF-007

Seu Desafio

Crie sua própria **matriz de rastreabilidade** conectando:

1. Todos os 7 requisitos funcionais
2. Os 10 casos de teste que você criou
3. Os resultados da execução (passou/falhou)
4. Os bugs registrados

Calcule:

- Cobertura de requisitos (%)
- Taxa de sucesso dos testes (%)
- Quantidade de bugs encontrados
- Requisitos com bugs críticos ou altos

Meta: Atingir pelo menos **80% de cobertura** é considerado bom para um teste inicial.

Entregáveis do Exercício

Ao final, você deverá ter criado os seguintes documentos:

1. Casos de Teste

- 10 casos de teste completos seguindo o template
- Cobertura de todos os requisitos (RF-001 a RF-007)
- Mix de testes positivos e negativos

2. Plano de Teste

- Escopo definido
- Abordagem e ferramentas documentadas
- Cronograma preenchido
- Critérios de aceite claros

3. Relatório de Execução

- Todos os 10 casos executados
- Status registrado (passou/falhou/bloqueado)
- Observações sobre comportamentos
- Evidências anexadas (opcional)

4. Registro de Bugs

- Pelo menos 1 bug registrado (se encontrado)
- Seguindo template completo
- Severidade e prioridade classificadas
- Passos para reproduzir detalhados

5. Matriz de Rastreabilidade

- Requisitos × Casos de Teste × Resultados
 - Cálculo de cobertura (%)
 - Análise de qualidade
-